Android UI Development

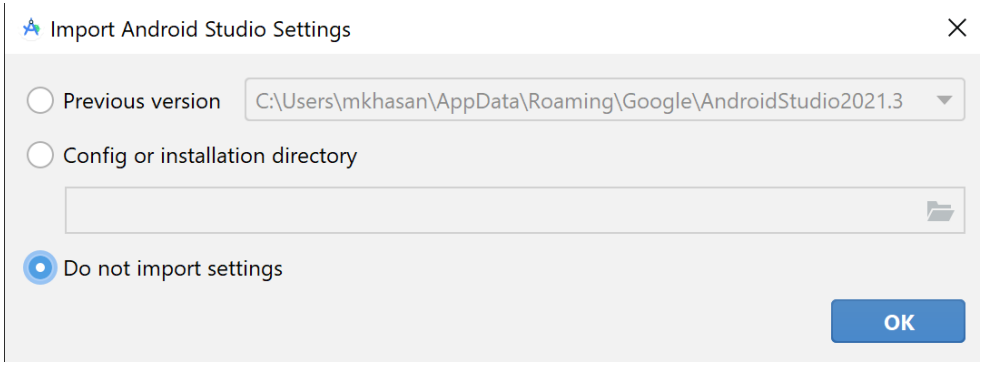
Android Studio

Widget

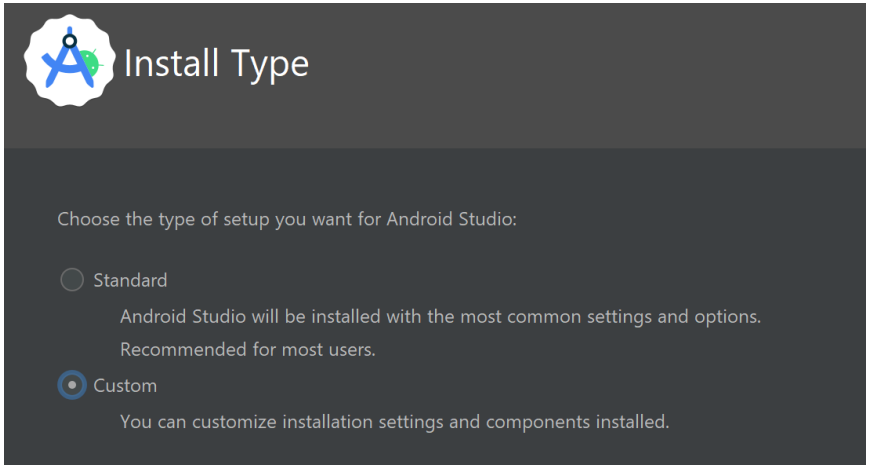
Layout

Android Development Environment

- Install Android Studio, and run it
- https://developer.android.com/studio
- Import previous settings

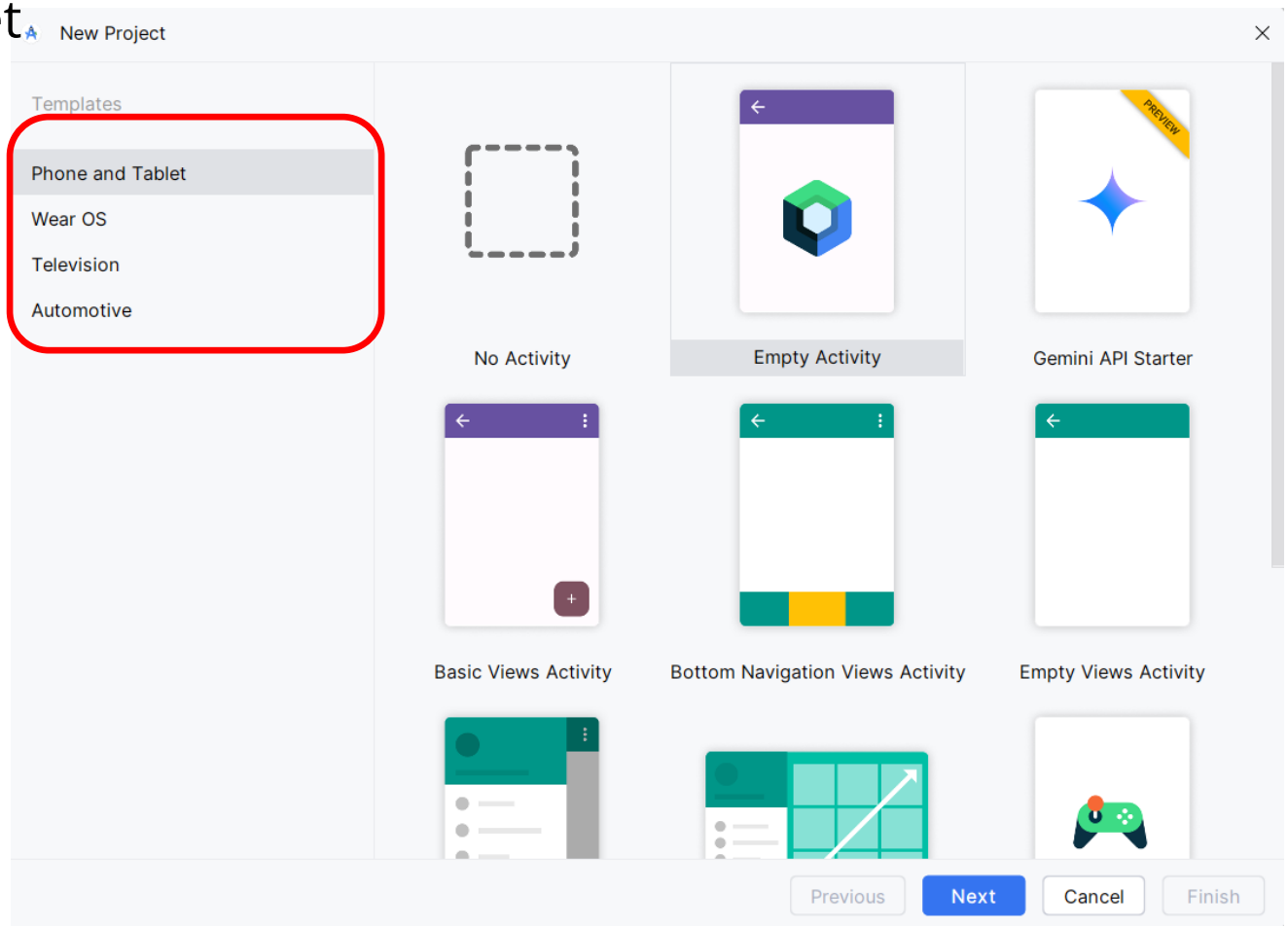


- Choose “standard” Install Type



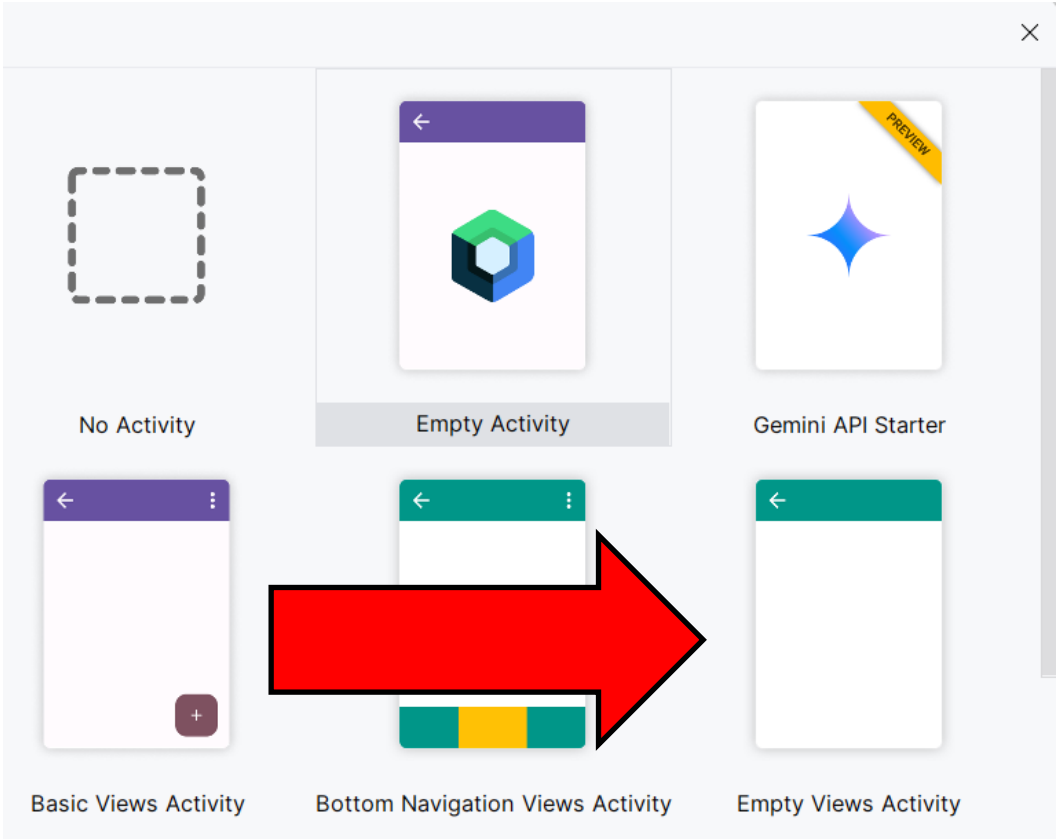
Create new project

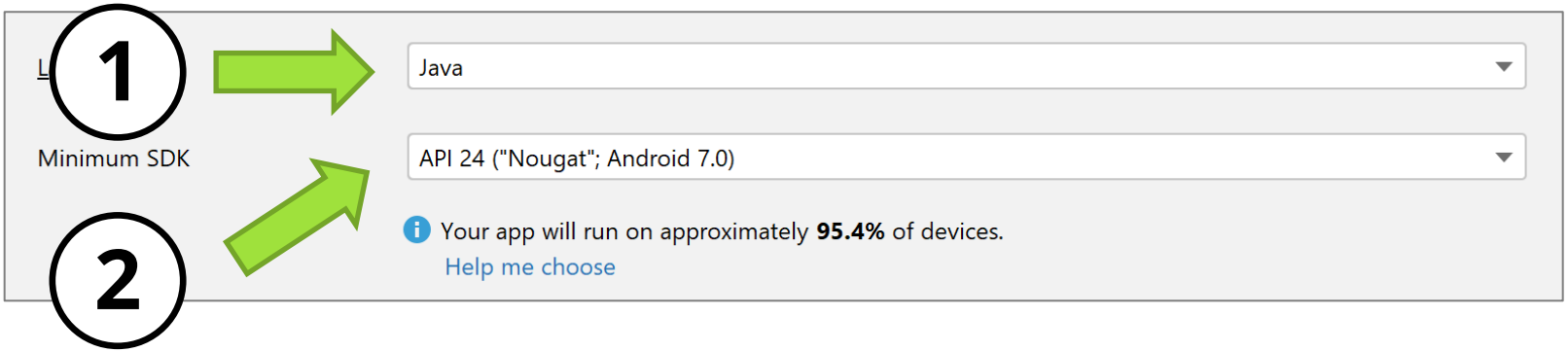
- Phone and Tablet
- Wear OS
  - Wearables
- Televisions
  - TV devices
  - Big screens
- Automotive



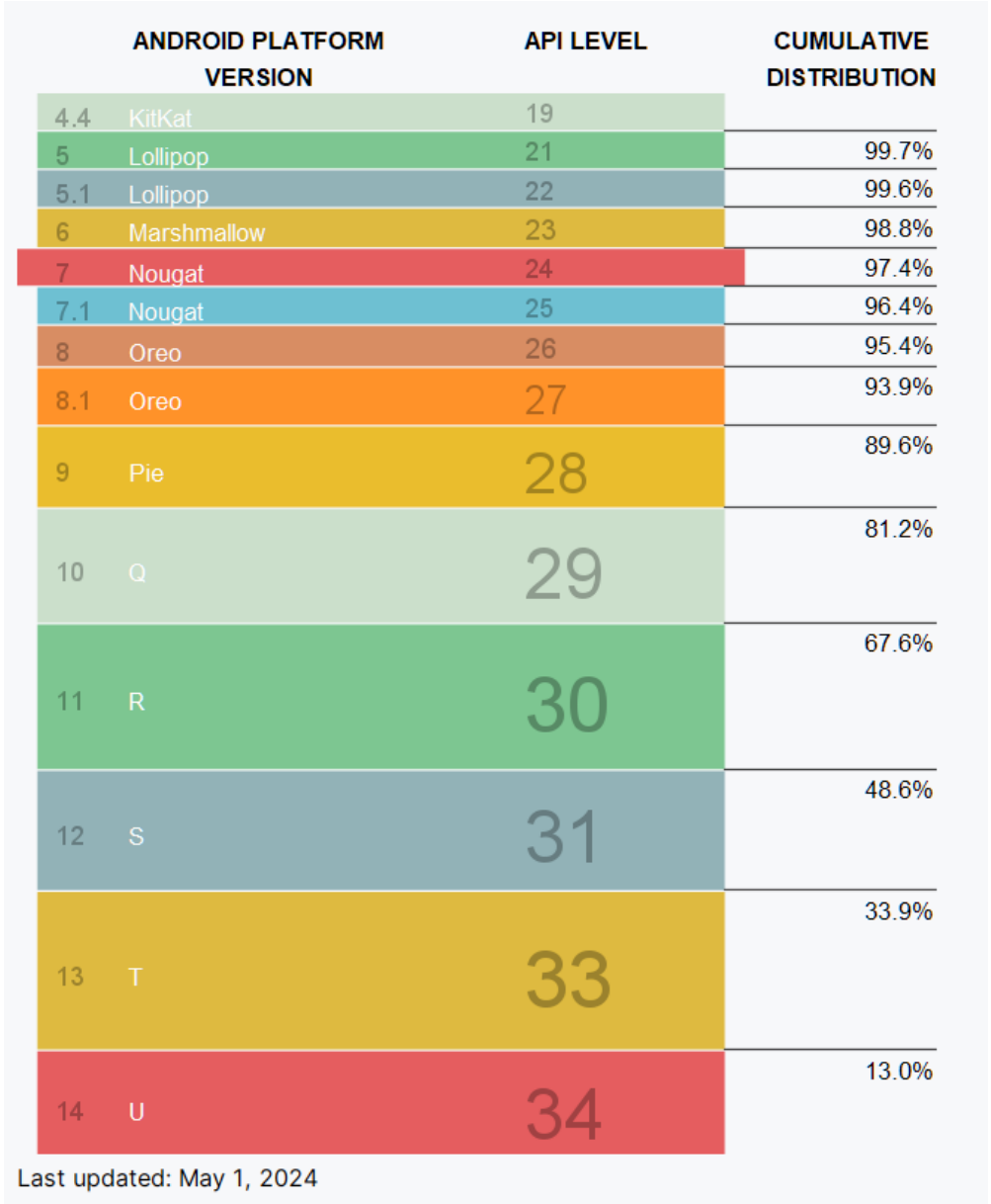
Create new project

- Phone and Tablet
  - Empty View Activity





Create new project

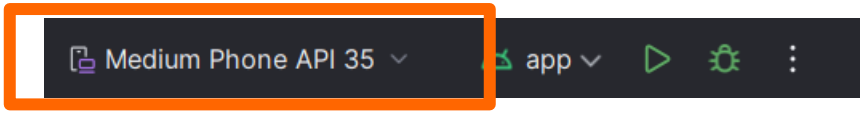


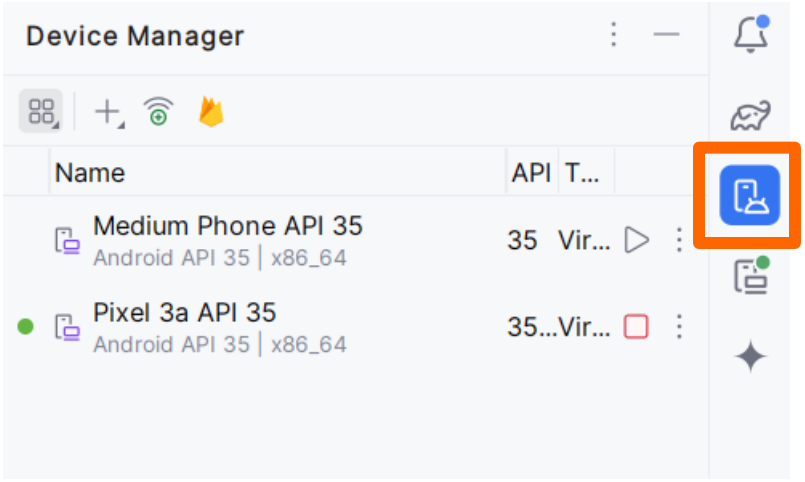
ANDROID PLATFORM VERSION		API LEVEL	CUMULATIVE DISTRIBUTION
4.4	KitKat	19	
5	Lollipop	21	99.7%
5.1	Lollipop	22	99.6%
6	Marshmallow	23	98.8%
7	Nougat	24	97.4%
7.1	Nougat	25	96.4%
8	Oreo	26	95.4%
8.1	Oreo	27	93.9%
9	Pie	28	89.6%
10	Q	29	81.2%
11	R	30	67.6%
12	S	31	48.6%
13	T	33	33.9%
14	U	34	13.0%

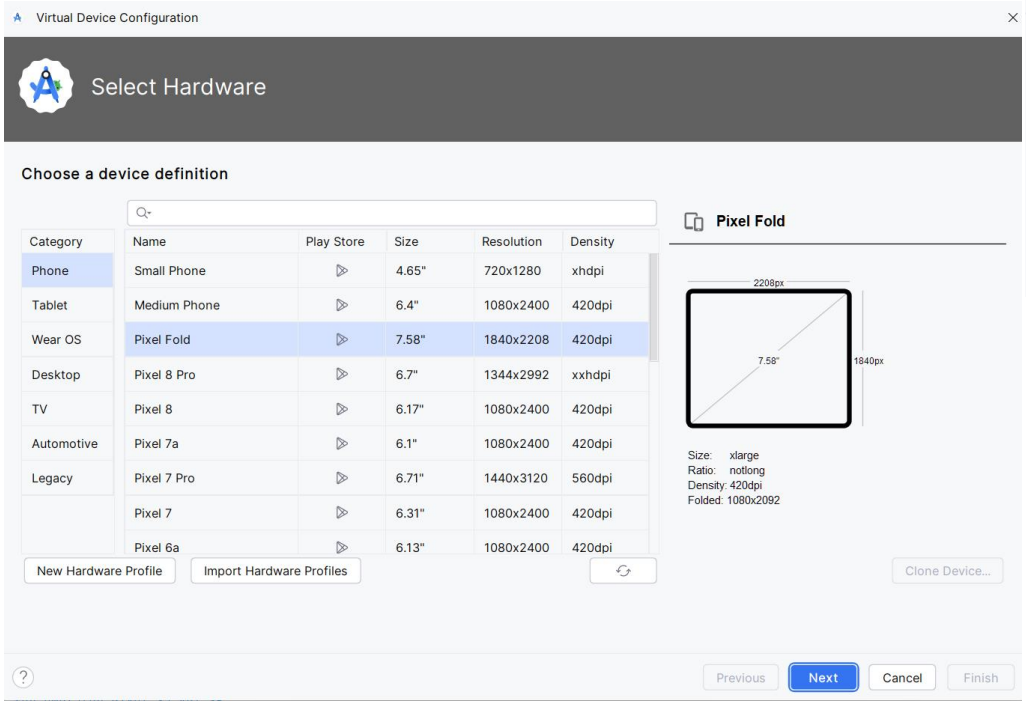
Last updated: May 1, 2024

Create Android Virtual Device (ADV) For testing

- The AVD contains the full Android software stack, and it runs as if it were on a physical device.

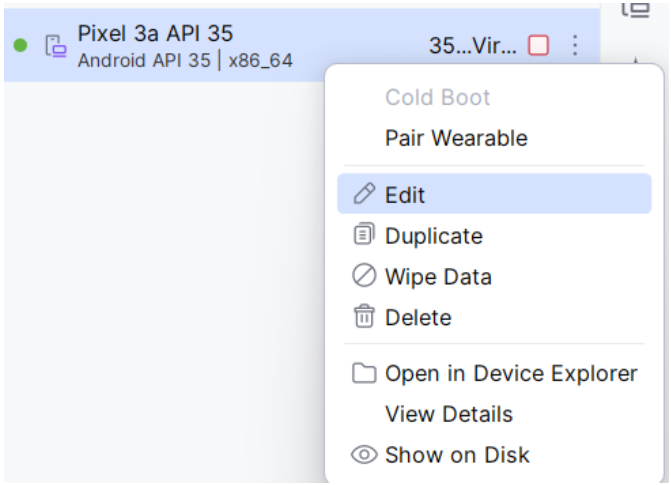


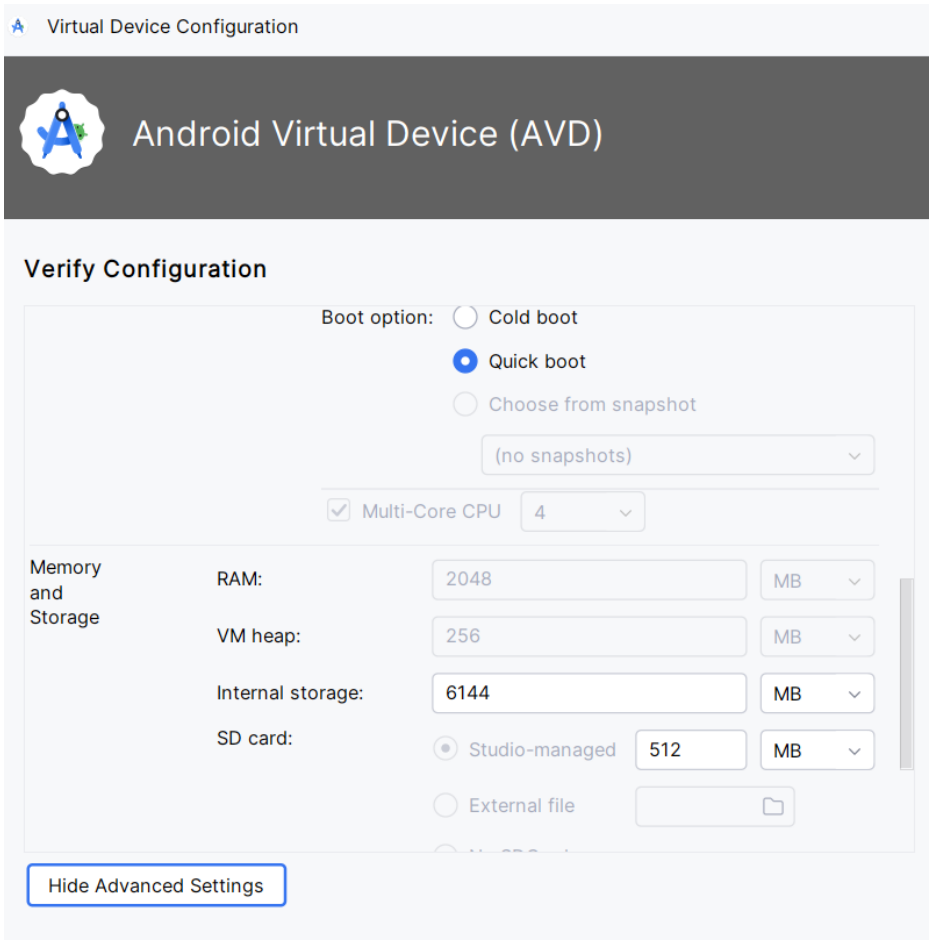




Android Virtual Device (AVD)

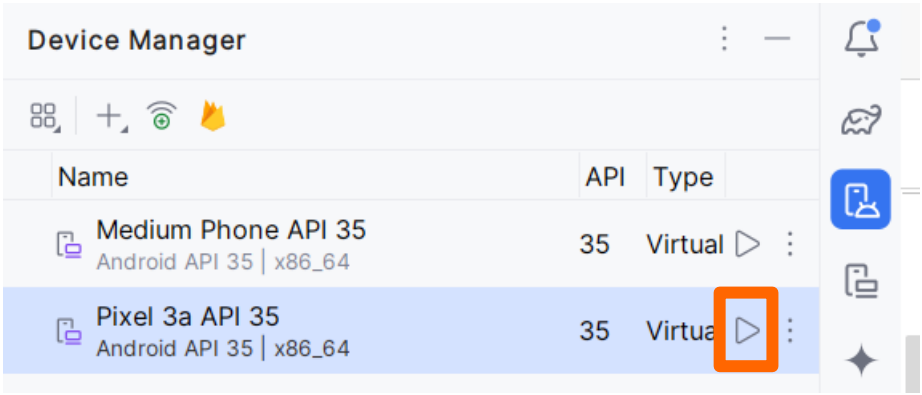
- The AVD is slow to launch, so keep it running in the background while you're programming / debugging.

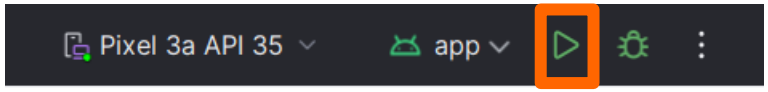


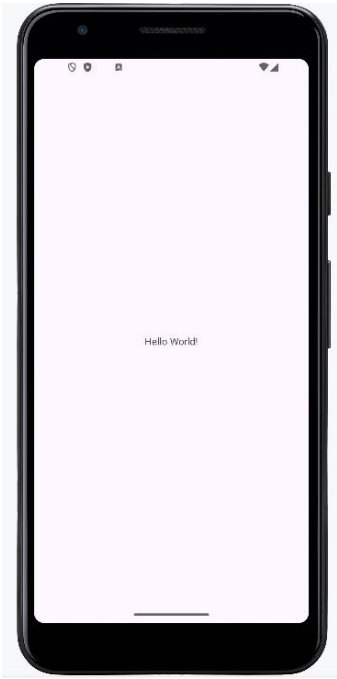


Run sample code

- Run
- Pixel 3a API 35

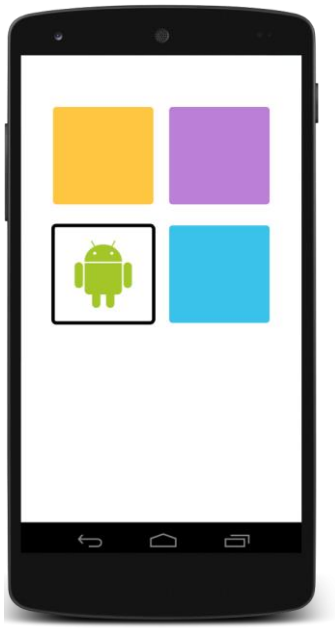




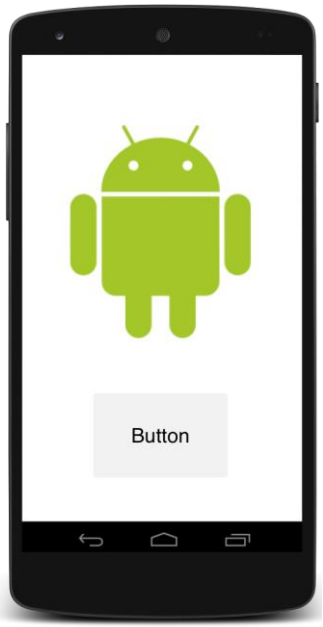


Activities

- A standard application component is an Activity
  - Typically represents a single screen
  - Main entry point (equivalent to main() method)
  - For most purposes, this is your application class



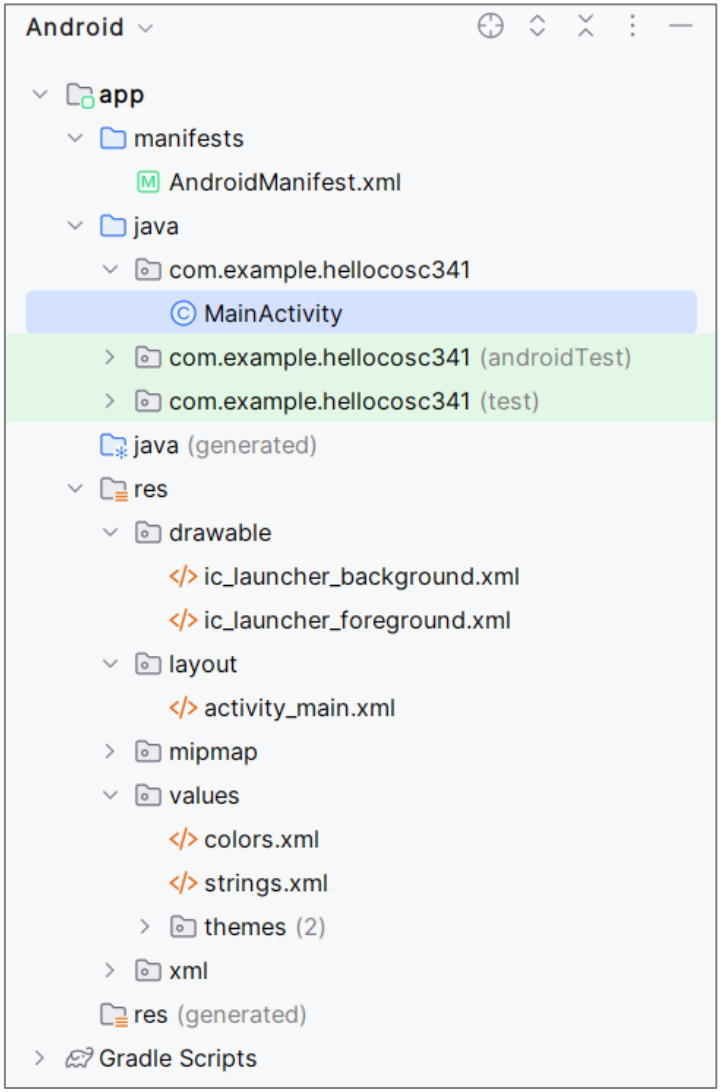
Activity 1



Activity 2

Android Project Files

- Manifest (app/manifests/)
  - Application setting
- Java (app/java/)
  - **(*.java) source code**
- Resources (app/res/)
  - **layout: (*.xml) UI layout and View definitions**
  - **values: (*.xml) constants** like strings, colours, ...
  - also bitmaps and SVG images (mipmap*, drawable*, ...)



Manifest

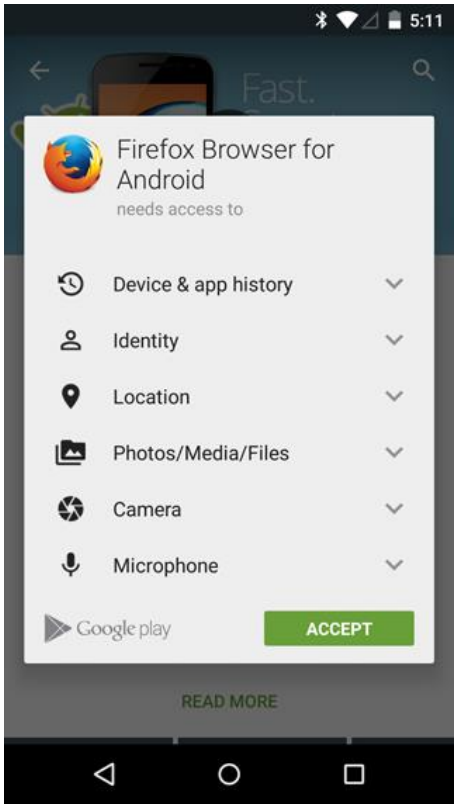
- Every app project must have an AndroidManifest.xml file
- Metadata about the app
- App components, Intent filters

```
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
        ...
        >
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

Manifest – Permissions -

```
<manifest>
...
<uses-permission
        android:name="android.permission.SEND_SMS" />
</manifest>
```

- Android must request permission to access sensitive user data



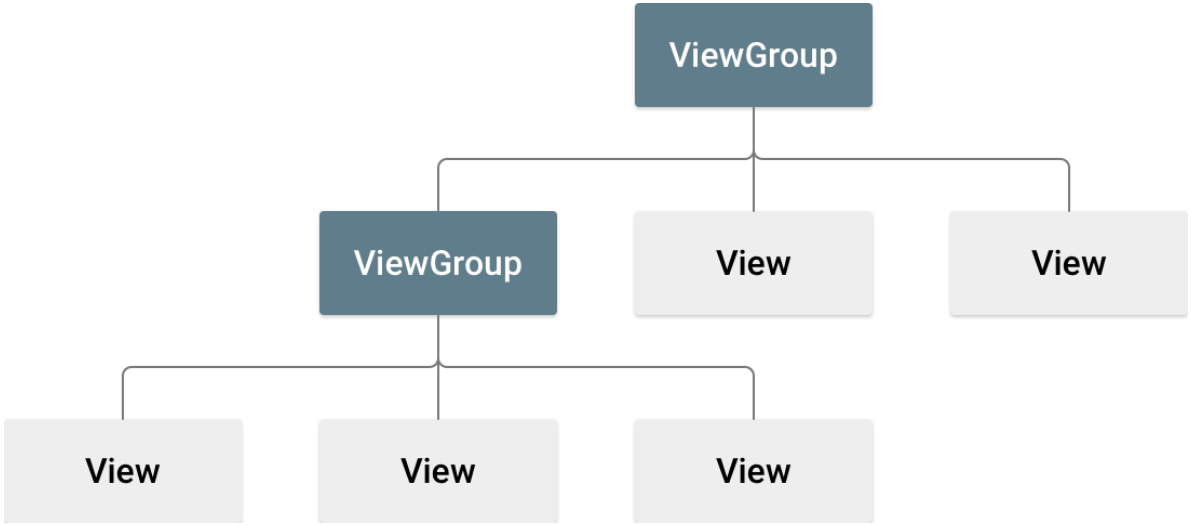
App Resources

- Each type of resource in a specific subdirectory of your project's res/ directory
- Access them using resource IDs that are generated in the project's R class

```
app/
        manifest/
        java/
        res/
                drawable/
                                graphic.png
                layout/
                                activity_main.xml
                mipmap/
                                icon.png
                values/
                                strings.xml
```

Layouts

- A layout defines the structure for a user interface
- All elements in the layout are built using a hierarchy of View and ViewGroup
- A View usually draws something the user can see and interact with
- A ViewGroup is an invisible container that defines the layout structure for View and other ViewGroup objects



https://developer.android.com/develop/ui/views/layout/declaring-layout

Views (android.widget)

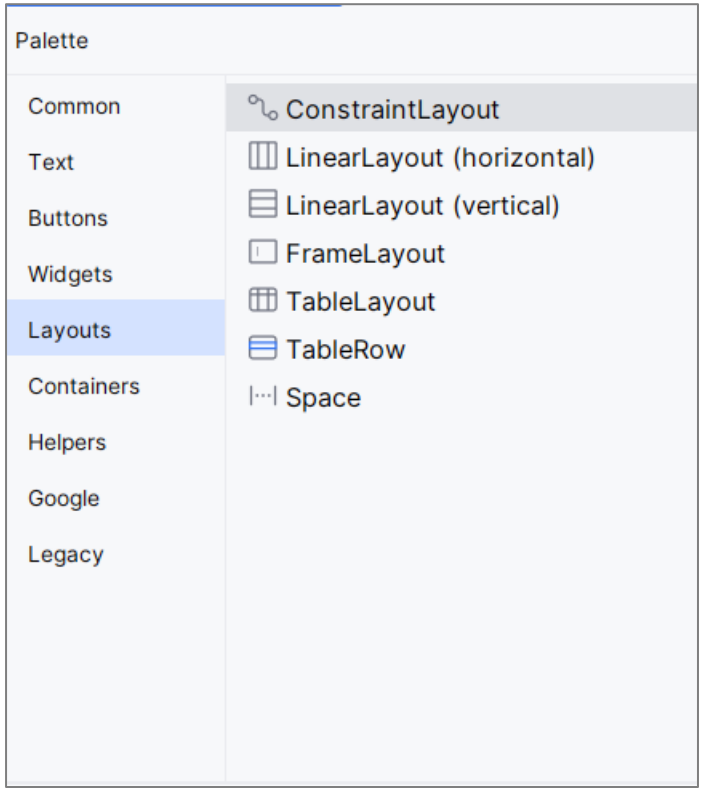
- The widget package contains (mostly visual) UI elements
- Subclasses:

```
android.widget.Button
android.widget.ImageView
android.widget.ProgressBar
Android.widget.TextView
...
```

https://developer.android.com/reference/android/widget/package-summary

ViewGroup

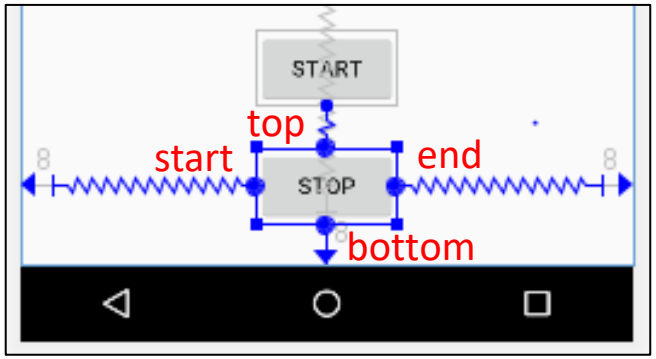
- The ViewGroup objects are usually called "layouts“
- A ViewGroup can be one of many types that provide a different layout structure, such as LinearLayout or ConstraintLayout



layout\activity_main.xml

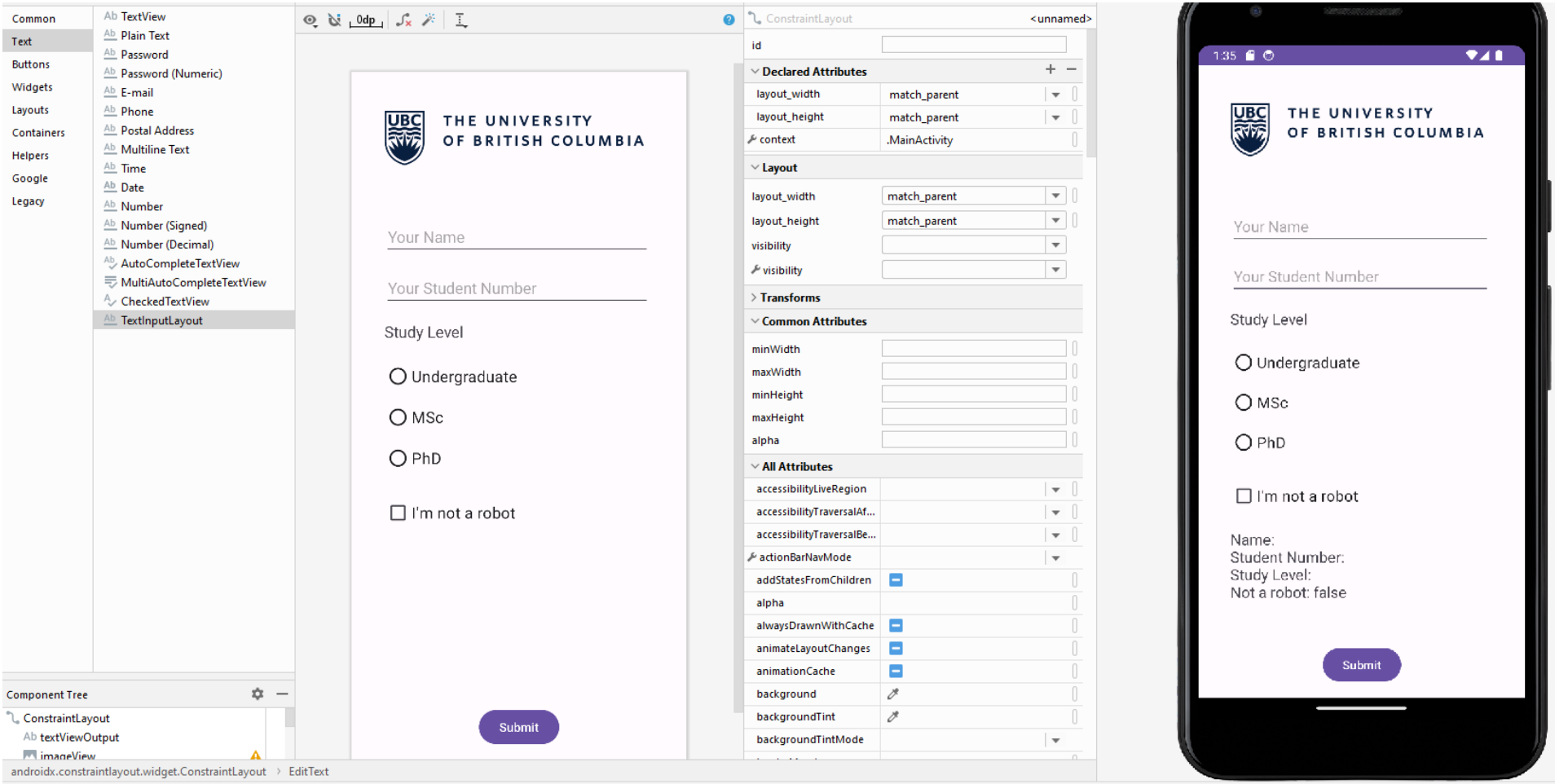
ConstraintLayout

- All views are laid out according to relationships with others t
- At least one **horizontal** and one **vertical** constraint



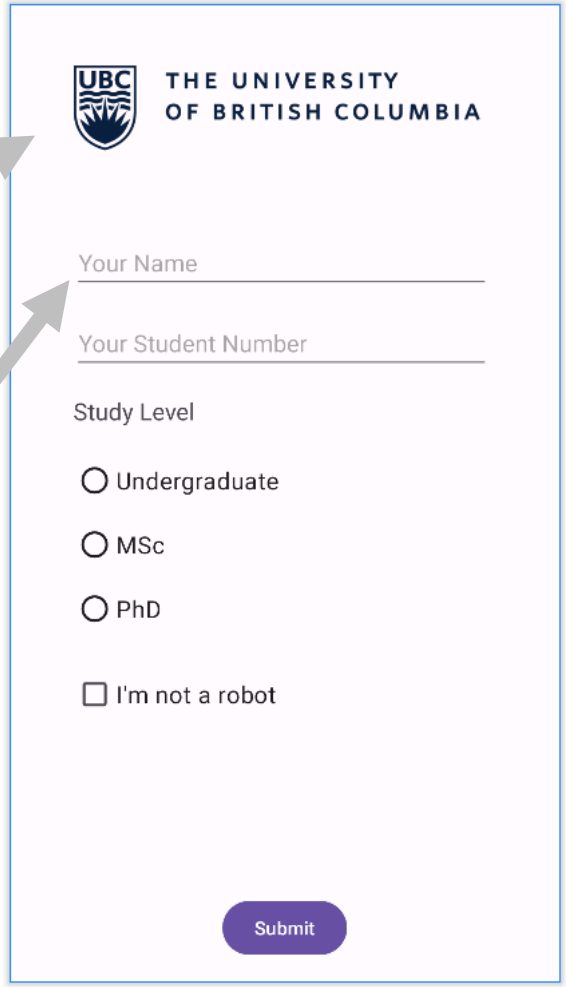

```
<Button
    android:id="@+id/stopButton"
    …
    android:text="Stop"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/startButton"
    app:layout_constraintBottom_toBottomOf="parent"   />
```

Editing XML


Common
Text
Buttons
Widgets
Layouts
Containers
Helpers
Google
Legacy
TextView
Plain Text
Password
Password (Numeric)
E-mail
Phone
Postal Address
Multiline Text
Time
Date
Number
Number (Signed)
Number (Decimal)
AutoCompleteTextView
MultiAutoCompleteTextView
CheckedTextView
TextInputLayout
Component Tree
ConstraintLayout
textViewOutput
imageView
THE UNIVERSITY OF BRITISH COLUMBIA
Your Name
Your Student Number
Study Level
Undergraduate
MSc
PhD
I'm not a robot
Submit
ConstraintLayout
<unnamed>
id
Declared Attributes
layout_width
match_parent
layout_height
match_parent
context
.MainActivity
Layout
visibility
Transforms
Common Attributes
minWidth
maxWidth
minHeight
maxHeight
alpha
All Attributes
accessibilityLiveRegion
actionBarNavMode
addStatesFromChildren
alpha
alwaysDrawnWithCache
animateLayoutChanges
animationCache
background
backgroundTint
backgroundTintMode
androidx.constraintlayout.widget.ConstraintLayout
EditText
1:35
Name:
Student Number:
Study Level:
Not a robot: false

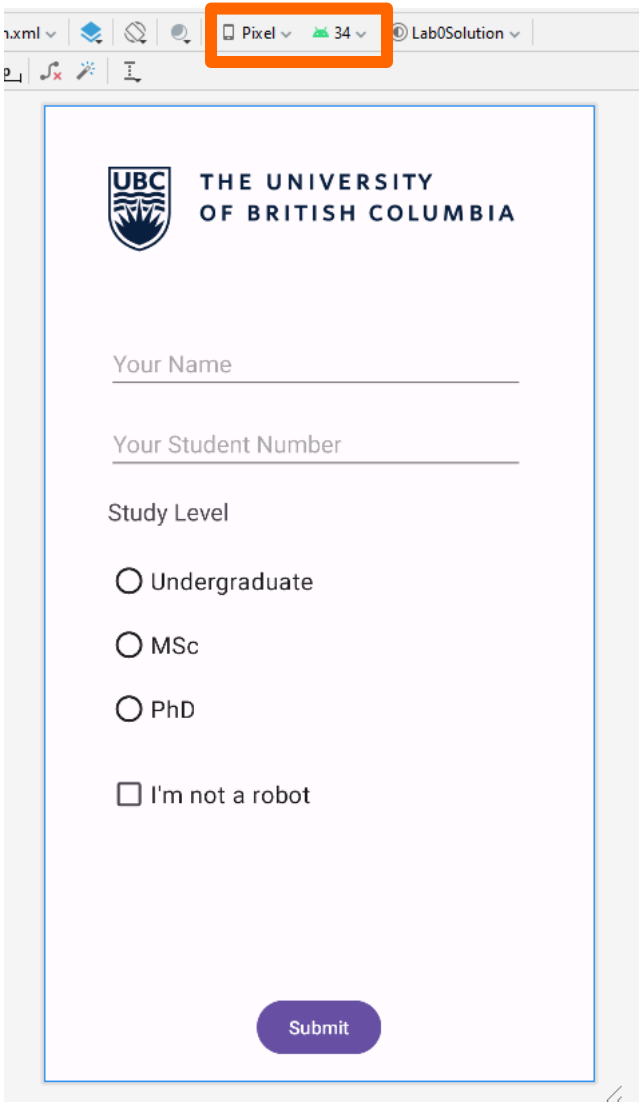
Layout Example

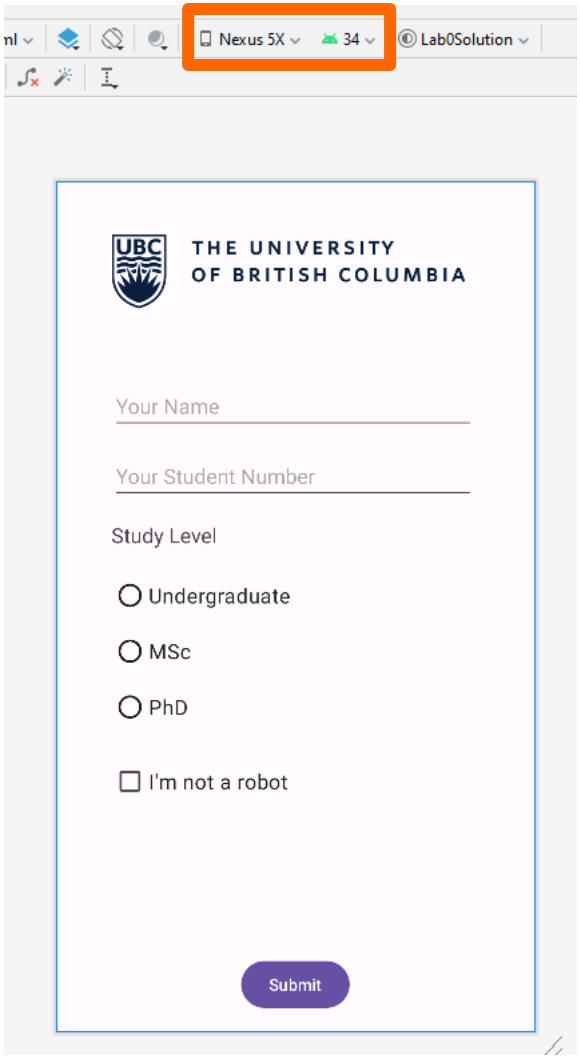
```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout
    …
    tools:context=".MainActivity">

<ImageView
    android:id="@+id/imageView"
    android:layout_width="316dp"
    android:layout_height="124dp"
    android:layout_marginTop="16dp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:srcCompat="@drawable/ubclogo" />

<EditText
    android:id="@+id/editTextName"
    android:layout_width="312dp"
    android:layout_height="44dp"
    android:layout_marginTop="32dp"
    android:ems="10"
    android:hint="Your Name"
    android:inputType="text"
    app:layout_constraintStart_toStartOf="@+id/imageView"
    app:layout_constraintTop_toBottomOf="@+id/imageView" />

</android.support.constraint.ConstraintLayout>
```



Layout test

- Check the layout with multiple screen sizes





Layout

- When you compile your app, each XML layout file is compiled into a View resource
- Activity class takes care of creating a window for you in which you can place your UI with `setContentView(View)`
- Calling `setContentView()`, passing it the reference to your layout resource in the form of: `R.layout.layout_file_name`.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
}
```

View (Widget)

Properties:

- Background color, text, font, alignment, size, padding, margin, etc

Event Listeners :

- Respond to various events such as: click, long-click, focus change, etc.

Set focus:

- Set focus on a specific view `requestFocus()` or use XML tag `<requestFocus />`

Visibility:

- You can hide or show views using `setVisibility(…)`.

Views: TextViews


```
<TextView
    android:id="@+id/txtHello"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />
```

layout: (*.xml)

```
TextView helloTextView = findViewById(R.id.txtHello);
helloTextView.setText("COSC 341");
```

Java (app/java/) source code

OR

```
TextView helloTextView = (TextView) findViewById(R.id.txtHello);
helloTextView.setText("COSC 341");
```

Views: EditText (Plain Text)


```
<EditText
    android:id="@+id/name"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content”
    android:inputType="text"
    android:text=”@string/name” >
    <requestFocus/>
 <EditText/>
```

layout: (*.xml)

```
EditText nameView = findViewById(R.id.name);
String name = nameView.getText().toString();
```

Java (app/java/) source code

Views: Buttons


```
<Button
    android:id="@+id/btnAlarm"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/alarm" />
```

layout: (*.xml)

https://developer.android.com/develop/ui/views/components/button

Responding to Button Events

- When the user clicks a button, a Button object receives an **on-click** event.
- To define the click event handler for a button, add the **`android:onClick`** attribute to the `<Button>` element in your XML layout.
- The value for this attribute must be the name of the method you want to call in response to a click event.

```
<Button
    android:id="@+id/btnAlarm"
    ……
    android:onClick="sendMessage"/>
```

mplement the corresponding

layout: (*.xml)

```
/** Called when the user touches the button */
public void sendMessage(View view) {
    // Do something in response to button click
}
```

Java (app/java/) source code

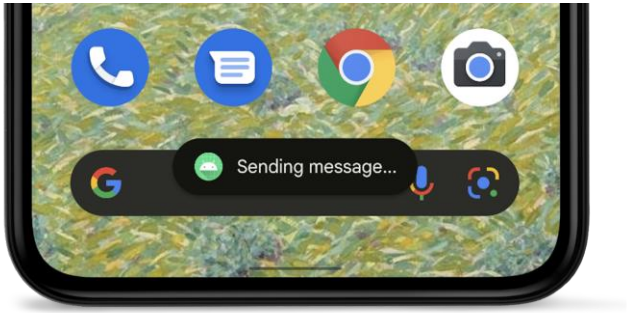
Responding to Button Events

- Alternative option
  - Within the Activity that hosts this layout, the following method handles the click event for the button:

```
Button button = (Button) findViewById(R.id.btnAlarm);
button.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        // Do something in response to button click
       Log.d("BUTTONS", "User tapped the button");
    }
});
```

Android Toast

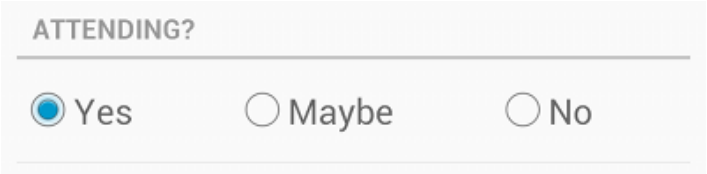
- A toast provides simple feedback about an operation in a small popup.
- It only fills the amount of space required for the message and the current activity remains visible and interactive.
- It only fills the amount of space required for the message and the current activity remains visible and intera




```
CharSequence text = "Sending message...";
int duration = Toast.LENGTH_SHORT;

Toast toast = Toast.makeText(this /* MyActivity */, text, duration);
toast.show();
```

Radio Buttons

- Radio buttons allow the user to select one option from a set.
- To create each radio button option, create a **RadioButton** in your layout.
- Radio buttons are mutually exclusive, you must group them together inside a **RadioGroup**.




```
<RadioGroup
    …
     android:orientation="horizontal">
     <RadioButton
        android:id="@+id/radio_yes"
        …
        android:onClick="onRadioButtonClicked"
        android:text="@string/yes" />
     <RadioButton
        android:id="@+id/radio_no"
        …
        android:onClick="onRadioButtonClicked"
        android:text="@string/no" />
</RadioGroup>
```

layout: (*.xml)

Radio Buttons

Within the Activity that hosts this layout:

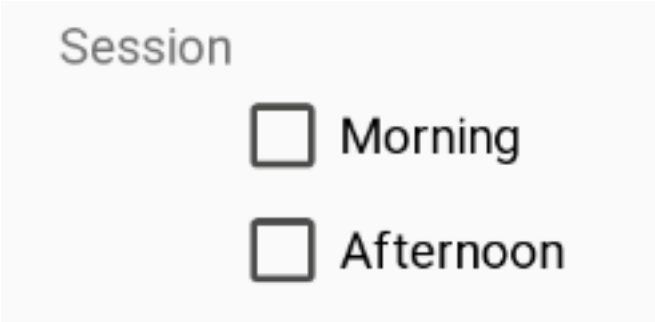
```
RadioGroup radioGroup = findViewById(R.id.radioGroup);
int selectedID = radioGroup.getCheckedRadioButtonId();
RadioButton radioButton = findViewById(selectedID);

Log.d("Toast", "Selected Item: " + radioButton.getText().toString());
```

Java (app/java/) source code

Checkboxes

A checkbox is a specific type of two-states button that can be either checked or unchecked.

```
<CheckBox
    android:id="@+id/checkbox_morning"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/morning" />

<CheckBox
    android:id="@+id/checkbox_afternoon"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/afternoon" />
```



https://developer.android.com/reference/android/widget/CheckBox

Checkboxes

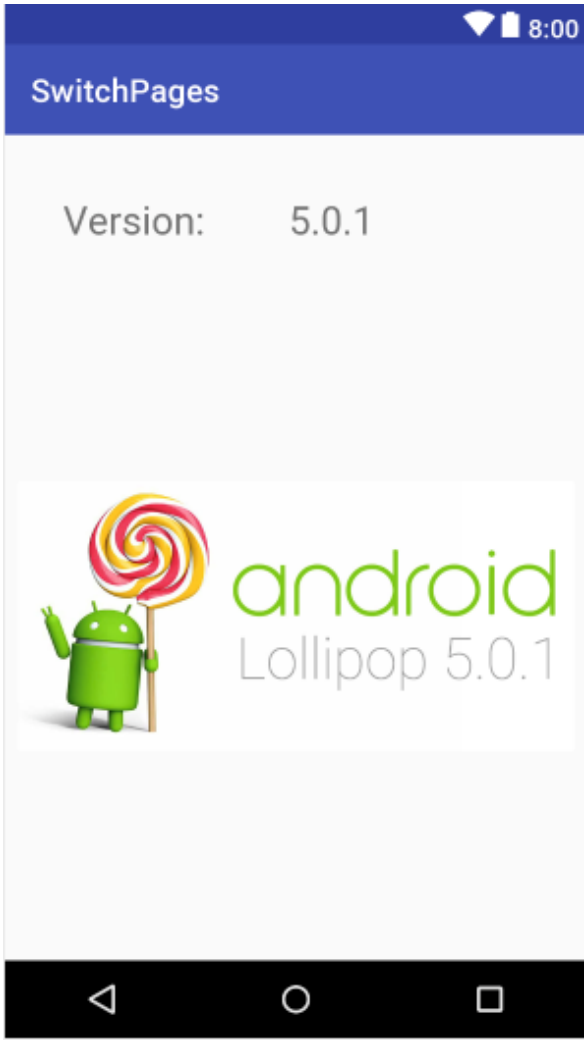
The following code goes into a button onClick event to show the check box names in a textview

```
CheckBox chkbox_morning =
                (CheckBox) findViewById(R.id.checkbox_morning);
CheckBox chkbox_afternoon =
                (CheckBox) findViewById(R.id.checkbox_afternoon);


txtView.setText("");

if (chkbox_morning.isChecked()) txtView.append("Morning");
if (chkbox_afternoon.isChecked()) txtView.append("Afternoon");
```

ImageView

- Displays image resources
  - Bitmap:
    the simplest Drawable, a PNG or JPEG
  - Drawable resources
    `Vector, Shape`

```
<ImageView
    android:id="@+id/imageView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content”
    android:srcCompat="@drawable/lollipop" />
```



Try it

- Notes
  - TextView
  - EditText
  - RadioButton
  - CheckBox
  - Button